Final Report

Namma Yatri

Submitted in partial fulfillment of the requirements for

Capstone Project

in

M.Tech Programme

by

Yakshith Maddineni (26080)
Vidhi Sonkar(25999)
Anant Rajput (27252)
Rahul Dev Sharma (26845)
Ashish Kumar Verma (26678)

Supervisor: **Prof. Vijay Kovvali**



Centre for infrastructure, Sustainable Transportation and Urban Planning

Indian Institute of Science, Bengaluru 560012

Contents

Table of Content **2**

1 Introduction **5**

2 Literature Review **8**
2.1 Prior Work on NY Share 8
2.2 Prior Work on NY Pool 8

3 NY Share **9**
3.1 Problem Formulation 9
3.2 Spatial Representation using H3 10
3.3 Neighbourhood Demand and Supply Estimation 11
3.4 Dynamic Waiting Time Window 11
3.4.1 Waiting Time Formulation 11
3.4.2 Base Waiting Time 12
3.4.3 Distance Adjustment 12
3.4.4 Time-of-Day Adjustment 12
3.4.5 Demand Density Adjustment 13
3.4.6 Supply Adjustment 13
3.4.7 Role of Dynamic Waiting Time in the Algorithm 13
3.5 Justification of Parameter Choices 13
3.5.1 Choice of Base Waiting Time (3, 5, 7 minutes) 14
3.5.2 Why Not Intermediate Values (e.g., 4 or 6 minutes)? 14
3.5.3 Clipping Bounds (3 to 8 minutes) 14
3.5.4 Distance Adjustment (1 minute per 5 km) 15
3.5.5 Demand Density Thresholds 15
3.5.6 Supply (ETA) Thresholds 16
3.5.7 Key Insight 16
3.6 Spatio-Temporal Matching Engine 17
3.6.1 Matching Procedure 17
3.7 Data Structures 17
3.8 Computational Complexity 18

4 NY Pool **20**
4.1 Problem Formulation 20
4.1.1 Ride Representation 20
4.1.2 Active Ride Set 21
4.1.3 The Dynamic Pooling Problem 21
4.1.4 Objective 21

4.1.5 Distinction from Static Batching . . . 22
4.2 Spatial Indexing using H3 . . . 22
4.2.1 Trajectory Encoding . . . 22
4.2.2 Grid Disk Neighbourhoods . . . 22
4.3 Core Data Structures . . . 22
4.3.1 Temporal Priority Queue . . . 22
4.3.2 Spatial Inverted Index . . . 23
4.3.3 Ride Metadata Cache . . . 23
4.4 Computational Subroutines . . . 23
4.4.1 Bearing Computation . . . 23
4.4.2 Directional Compatibility . . . 24
4.4.3 Trajectory Similarity Score . . . 24
4.4.4 Road-Network Detour Validation . . . 25
4.5 The Sweep-Line Streaming Algorithm . . . 26
4.5.1 File Discovery and Loading Strategy . . . 26
4.5.2 The Sweep-Line Event Loop . . . 26
4.5.3 Ride Lifecycle and Status Handling . . . 26
4.5.4 Expiry and Eviction . . . 27
4.5.5 Matching Attempt Throttling . . . 27
4.5.6 Hierarchical Filtering Pipeline . . . 27
4.6 Computational Complexity and Memory Analysis . . . 29
4.6.1 Time Complexity . . . 29
4.6.2 Memory Complexity . . . 29
4.7 Design Choices and Limitations . . . 30
4.7.1 Key Design Decisions . . . 30
4.7.2 Limitations and Potential Improvements . . . 30

5 Analysis of NY Share and NY Pool **32**
5.1 Analysis of NY Share . . . 32
5.2 Analysis of NY Pool . . . 34
5.3 VKT Analysis (Vehicle Kilometers Traveled) . . . 35
5.3.1 VKT—NY Share . . . 35
5.3.2 VKT—NY Pool . . . 36

6 Conclusion **38**

References **39**

List of Figures

1.1 Traffic congestion at peak hour in Bengaluru. 5
1.2 Projection of Indian two wheeler market. 6
1.3 PM-eSEWA Bus Scheme. 6
1.4 Projected growth of India's Shared Mobility market. 7

3.1 Architecture of the spatio–temporal ride batching system. 10
3.2 Comparison of Allowed Maximum Wait Distribution and Actual Wait Distribution for Base Time = 3 minutes. 18
3.3 Comparison of Allowed Maximum Wait Distribution and Actual Wait Distribution for Base Time = 5 minutes. 19
3.4 Comparison of Allowed Maximum Wait Distribution and Actual Wait Distribution for Base Time = 7 minutes. 19

4.1 Hierarchical filtering pipeline for dynamic pooling candidate selection. 29

5.1 Density of batched rides for 14-days. 32
5.2 Hourly distribution of batched rides for 14-days. 32
5.3 Density of batched rides for August 2, 2025 34
5.4 Hourly distribution of batched rides for August 2, 2025 34
5.5 Percentage of distance saved for NY share 36
5.6 Percentage of distance saved for NY Pool 36

List of Tables

3.1 Batching performance across base waiting times 14

4.1 Global data structures maintained during the sweep-line streaming phase. . . 23
4.2 Batching performance of NY pool on 2nd Aug 29
4.3 Per-event time complexity of key operations. 30

5.1 Emission Summary (in kg) for NY Share 35
5.2 Emission Summary (in kg) for NY Pool 37

Chapter 1

Introduction

At a time when urban planning conventionally prioritized automobiles, the urbanist Jane Jacobs argued in her classic *The Death and Life of Great American Cities* that cities should move away from being fully car centric, not by eliminating cars but by rebalancing priorities. She argued that congestion arises from the dominance of vehicles rather than from people themselves and noted that pre automobile congestion caused by horse buggies was later replicated by motor cars. She instead advocated for stronger public transportation systems and more pedestrian friendly urban design. Her ideas remain relevant as seen in Paris in 2024 where cars were restricted within a 5.18 sq km zone to promote pedestrianisation. Similarly, the Government of India's Smart Cities Mission seeks to create vibrant public spaces and improve mobility through Non Motorized Transport, expansion of public transport, and multimodal integration. Yet, despite these efforts, congestion continues.



Figure 1.1: Traffic congestion at peak hour in Bengaluru.

Traffic congestion has become an unavoidable reality across India's Tier-I and Tier-II cities, and understanding its root causes is essential to addressing it meaningfully. At its core, the problem stems from a fundamental imbalance: road infrastructure that has remained relatively static is being asked to absorb a rapidly expanding vehicle population. This imbalance is not marginal between 2010 and 2020, the total number of registered vehicles in million-plus cities surged from 35.6 million to 113.25 million, representing a staggering 218% increase in just one decade.

What makes this growth particularly consequential for urban road capacity, is its composition? According to the *Road Transport Year Book of 2020*, non-transport vehicles specifically cars and two-wheelers accounted for 86.7% of this pool, meaning the vast majority of vehicles on the road serve individual rather than collective mobility needs, consuming disproportionately more road space per passenger. This trend shows no signs of reversing, projections indicate that the Indian two-wheeler market alone will reach 92 million units in annual sales by 2033(see Figure 1.2). Taken together, these figures paint a clear picture: without a structural shift in how Indians move through cities, congestion will not merely persist it will deepen.

To accommodate additional traffic, cities are supposed to push seriously for public transport enhancement and integration (expanding catchment routes of metro/suburban rail and bus as well as increasing their capacity to cater to the expected induced ridership). Rather, we have been on a flyover building spree creating more congestion channels. With no offence to the engineering marvels that are achieved, the real question is, do they resolve

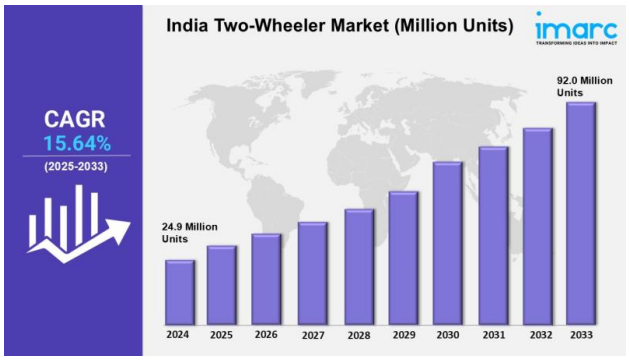


Figure 1.2: Projection of Indian two wheeler market.

congestion problems? While some cities like Bengaluru and Mumbai are scaling up their metro rail network, the urban dynamics show that the capacity of trains are limited, necessitating a need to additionally focus on buses.

A snapshot of some efforts in that direction needs to be mentioned here for more context. Earlier, through the former flagship mission of the Government of India the *Jawaharlal Nehru National Urban Renewal Mission (JNNURM)* necessary funding and planning were provided for cities to scale up their public transport along with other urban infrastructure projects up until 2014. A number of cities saw an expansion of their bus fleets owing to this programme.

Years later, in a renewed push towards strengthening urban bus systems, the Government of India included public transport as a component of the *Smart Cities Mission*, along with allocating about Rs. 18,000 crore for bus procurement in the 2021–22 Union Budget. Subsequently, in 2023, the Union Cabinet approved the *PM-eSEWA Bus Scheme* with an outlay of Rs. 20,000 crore. Additionally, the push towards public transit has been largely focused on metro rail networks, which is certainly commendable. In 2002, only Delhi and Kolkata had operational metro rail systems; as of now, metro services are operational in 13 cities across India, with several cities continuing to expand their networks. However, does this fully address congestion problems?



Figure 1.3: PM-eSEWA Bus Scheme.

Delhi has the densest metro network in the country and still ranks among the most polluted cities, with significantly high levels of AQI as reported by *IQAir*. Despite having the second-largest metro network after the National Capital Region, Bengaluru ranks second among Indian cities as the most congested city according to the *TomTom Traffic Index*, which measures travel time over a fixed distance as a proxy for congestion levels. Cities such as Pune, Hyderabad, and Chennai follow, all reporting considerably high levels of congestion (while Delhi shows relatively lower congestion levels in comparison).

Is it ideal to encourage public transport and active travel, expecting an overnight utopian shift? Pessimistically, it might not be possible due to various challenges such as infrastructure, enforcement, politics, economic factors, etc. But what does one do with the existing private

vehicles? Vehicular ownership in Delhi alone, has witnessed a growth from 317 vehicle per 1,000 population in 2005-06, vehicular ownership increased to 598 vehicles per thousand population. According to the Delhi's economic survey 2018-19, there are 70 lakh two-wheelers in Delhi (Survey, 2019).

One of the most practical and impactful approaches to addressing modern urban transportation challenges is the promotion and adoption of shared mobility. Unlike purely theoretical frameworks, shared mobility is grounded in real and observable shifts in mobility patterns, driven by growing public awareness around environmental sustainability and urban congestion. This makes it not just a viable solution, but an increasingly advantageous one in the pursuit of sustainable mobility.

The rapid growth of accessible technology has been a key enabler in making shared mobility more efficient and scalable. By leveraging digital platforms, real-time data, and connectivity, shared mobility systems can now directly address the core problem of too many vehicles on the road countering vehicle proliferation with smarter vehicle utilization. Beyond resolving congestion and reducing pollution, shared mobility also delivers a broader range of benefits, including first-mile and last-mile connectivity, reduced transportation costs for users, and improved accessibility across urban and peri-urban areas.

Shared mobility encompasses a diverse range of modes and services, including bike-sharing and rentals, carpooling and vanpooling, car-sharing, on-demand transit services, micromobility solutions, and microtransit. This variety ensures that shared mobility can cater to different user needs, trip types, and urban contexts.

The concept of shared mobility continues to evolve, with emerging models such as NY Share and NY Pool representing the next generation of integrated shared transport solutions. These models form the central focus of this report.



Figure 1.4: Projected growth of India's Shared Mobility market.

Chapter 2

Literature Review

The design of both NY Share and NY Pool draws on prior work in online matching, ride-sharing optimisation, and fleet management, and this chapter surveys the key contributions that shaped each system.

2.1 Prior Work on NY Share

The design of NY Share builds on a body of work that has steadily advanced our understanding of delay and matching in ride-sharing systems. The starting point is [6], who showed mathematically that waiting before making a match is not a cost to be minimised blindly, it is, in fact, a tool. By holding a request open for a short window, the system gains access to a larger pool of candidates and can find a significantly better pairing. This insight is foundational. However, the model stops short of telling a real system *how long* to wait, the delay falls out of a global cost equation rather than being calculated from what is actually happening on the ground at that moment. [7] take a more practical angle, surveying the optimisation methods that power real-time ride-sharing systems and laying out the constraints, detour limits, time windows, scalability, that any deployable system must respect. Even so, waiting time in their framework is treated as a fixed boundary, something the system works within rather than actively adjusts. [8] bring the lens of economics to the problem, showing that imbalances between rider demand and driver supply have a measurable impact on how well a shared vehicle platform performs. Their approximation framework is rigorous and general, yet it reasons about the platform as a whole and does not reach down to the level of a single incoming request to ask how long *that specific trip* should wait before being matched. This is precisely the gap that NY Share is designed to fill.

2.2 Prior Work on NY Pool

Pooling sits within a parallel tradition that asks a different but equally important question specifically, given a full fleet and a city's worth of demand, how close to optimal can a ride-pooling system actually get? [9] answer this with striking clarity. By re-solving the entire assignment problem every 30 seconds, building a graph of feasible trip combinations and solving it as an integer programme, they show that just 3,000 four-seat vehicles can serve 98% of New York City's taxi demand, cutting the required fleet to a quarter of its original size. This result sets the performance ceiling that any pooling system aspires to approach. [10] had earlier established why that ceiling is so high, using 150 million NYC taxi trips, they demonstrated that at a delay tolerance of just 300 seconds, roughly half of all trips can be shared with at least one other passenger, and total vehicle distance can be cut by 40% or more. The opportunity for pooling is vast and the challenge is, capturing it efficiently in real time. [11] provide the formal language for doing so. Their treatment of the Dial-a-Ride Problem sets out the constraints, pickup and delivery windows, maximum ride times, vehicle capacity, that translate the pooling opportunity into a solvable operational problem. Together, these three works define the theoretical landscape within which NY Pool operates: a large, tractable pooling opportunity, a proven method for capturing it at scale, and a rigorous constraint vocabulary to keep solutions feasible and passenger-friendly.

Chapter 3

NY Share

The primary objective of the batching system is to identify ride requests that can be served together while maintaining acceptable passenger waiting times. In dense urban mobility networks, many trips originate from nearby locations and travel toward similar destinations within short time intervals. If these requests are detected early, they can be grouped into a shared trip, reducing the number of dispatched vehicles and improving system efficiency.

The batching algorithm designed in this work operates as a *spatio–temporal streaming matching system*. Ride requests are processed sequentially in chronological order, and each request is either matched with an existing compatible request or temporarily stored in an active buffer while awaiting potential matches.

The algorithm combines three key components:

1. Spatial discretization of trip locations using the H3 hexagonal grid
2. Context-aware estimation of local demand and supply
3. A dynamic waiting time mechanism that adapts to trip characteristics and system conditions

Together these components allow the system to efficiently identify compatible ride pairs while keeping passenger waiting times bounded.

3.1 Problem Formulation

Each ride request can be represented as a tuple

$$r = (p, d, t)$$

where

- p denotes the pickup location
- d denotes the drop-off location
- t denotes the request timestamp

The batching algorithm attempts to identify pairs of ride requests that satisfy three compatibility constraints:

1. **Pickup proximity:** The pickup locations of both requests must lie within a small geographic neighbourhood.
2. **Drop-off compatibility:** The drop-off locations must also be geographically close.
3. **Temporal compatibility:** Both rides must occur within an acceptable waiting window.

If these conditions are satisfied, the two requests can be served together as a single batched trip.

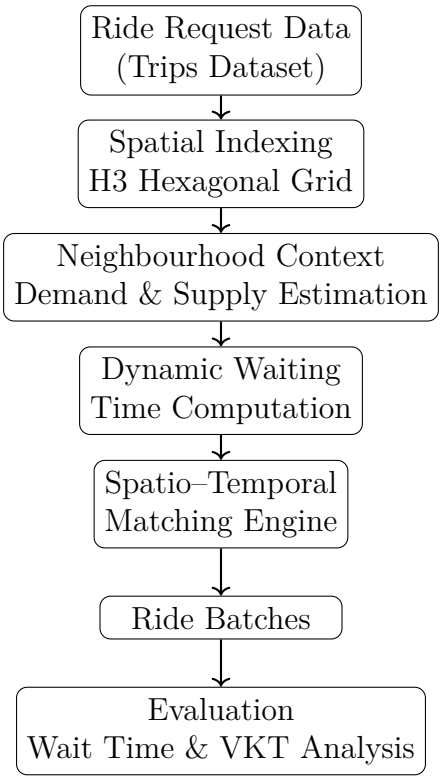


Figure 3.1: Architecture of the spatio–temporal ride batching system.

3.2 Spatial Representation using H3

To efficiently evaluate spatial proximity between ride requests, the system uses the H3 hexagonal hierarchical spatial indexing framework.

H3 partitions the Earth's surface into hexagonal cells at multiple resolutions. In this work, resolution 9 is used. At this resolution, each hexagon covers a small urban area with an edge length of approximately 170–200 meters. This spatial granularity is sufficient for identifying nearby ride requests while keeping the number of spatial partitions manageable.

For each ride request, two spatial indices are computed:

- Pickup hexagon h_p
- Drop-off hexagon h_d

Using H3 indexing enables efficient neighbourhood queries. Instead of computing geographic distances between all pairs of rides, the algorithm checks whether two rides belong to the same hexagon or to neighbouring hexagons.

For each hexagon h, a neighbourhood set is defined as

$$N(h) = \texttt{grid_disk}(h, 1)$$

which contains the hexagon itself and all immediately adjacent hexagons. These neighbourhoods are used to identify spatially compatible ride candidates.

3.3 Neighbourhood Demand and Supply Estimation

In addition to spatial location, the batching algorithm considers contextual information about the surrounding transportation environment.

Ride requests are grouped by pickup hexagon and hour of day to compute two statistics:

- **Neighbourhood demand:** number of ride requests originating from the region
- **Neighbourhood driver availability:** approximated by the median driver arrival time (ETA)

To reduce noise caused by sparse observations, these statistics are spatially smoothed. For a given pickup hexagon h and hour t, the algorithm aggregates demand and ETA across the neighbourhood $N(h)$:

$$D(h,t) = \sum_{n \in N(h)} \text{demand}(n,t)$$

$$E(h,t) = \text{median}_{n \in N(h)}(\text{ETA}(n,t))$$

where $D(h,t)$ represents neighbourhood demand and $E(h,t)$ represents neighbourhood driver availability. These contextual features are later used to adjust the dynamic waiting window.

3.4 Dynamic Waiting Time Window

A critical component of the batching algorithm is determining how long a ride request should remain in the matching pool before being served individually. A short waiting window limits batching opportunities, while an excessively long waiting window can increase passenger waiting time and degrade user experience.

To balance these competing objectives, the system assigns each ride request a **dynamic maximum waiting time**. This value is calculated using a base waiting time and several contextual adjustment factors derived from trip characteristics and neighbourhood conditions.

3.4.1 Waiting Time Formulation

The raw waiting window for a ride request is computed as

$$W_{\text{raw}} = W_{\text{base}} + F_{\text{distance}} + F_{\text{time}} + F_{\text{density}} + F_{\text{supply}}$$

where

- W_{base} is the base waiting time parameter
- F_{distance} adjusts waiting time based on trip distance
- F_{time} adjusts waiting time based on time of day
- F_{density} adjusts waiting time based on neighbourhood demand
- F_{supply} adjusts waiting time based on driver availability

The final waiting window is bounded as

$$W = \min(8, \max(3, W_{\text{raw}}))$$

ensuring that waiting times remain within the range of 3 to 8 minutes.

3.4.2 Base Waiting Time

The base waiting time W_{base} represents the minimum waiting duration assigned to each ride request before contextual adjustments are applied.

To study the effect of different waiting thresholds on batching performance, three base values are evaluated:

- 3 minutes
- 5 minutes
- 7 minutes

These values serve as the starting point for the dynamic waiting time calculation.

3.4.3 Distance Adjustment

Longer trips generally provide greater opportunity for shared rides because the potential detour cost can be amortized over a larger travel distance.

To capture this effect, the waiting window increases with trip distance. The adjustment factor is defined as

$$F_{\text{distance}} = \min\left(\frac{\text{chargeable distance}}{5000}, 4\right)$$

where distance is measured in meters. This formulation increases the waiting window by approximately one minute per five kilometres of trip distance, capped at a maximum increase of four minutes.

3.4.4 Time-of-Day Adjustment

Ride demand varies significantly across different hours of the day. During peak commuting hours, ride requests are more frequent and matching opportunities occur more rapidly.

To reflect this effect, the waiting window is reduced by one minute during peak periods. In the current implementation, peak hours are defined as:

- 08:00–10:00
- 17:00–19:00

Formally,

$$F_{\text{time}} = \begin{cases} -1 & \text{if } t \in \text{peak hours} \\ 0 & \text{otherwise} \end{cases}$$

This adjustment encourages faster matching during high-demand periods.

3.4.5 Demand Density Adjustment

The probability of finding a compatible ride partner depends strongly on local request density. In regions with high demand, matches are likely to appear quickly, allowing the waiting window to be reduced.

Neighbourhood demand is estimated by aggregating requests within the spatial neighbourhood of the pickup hexagon.

The density adjustment is defined as

$$F_{\text{density}} = \begin{cases} -1 & \text{if } D > 50 \\ 2 & \text{if } D < 10 \\ 0 & \text{otherwise} \end{cases}$$

where D represents neighbourhood demand measured in ride requests per hour.

Thus, densely populated regions reduce waiting time, while sparse areas extend the waiting window to increase matching opportunities.

3.4.6 Supply Adjustment

Driver availability also influences batching behaviour. When driver supply is low, passengers may tolerate slightly longer waits if it increases the likelihood of ride pooling.

Driver availability is approximated using the neighbourhood median driver arrival time (ETA).

The supply adjustment is defined as

$$F_{\text{supply}} = \begin{cases} 2 & \text{if } ETA > 600 \text{ seconds} \\ -1 & \text{if } ETA < 180 \text{ seconds} \\ 0 & \text{otherwise} \end{cases}$$

Longer arrival times increase the waiting window, while highly available driver supply reduces it.

3.4.7 Role of Dynamic Waiting Time in the Algorithm

The dynamic waiting window determines how long a ride request remains eligible for batching.

When a ride request arrives at time t, its expiration time is computed as

$$t_{\text{expiry}} = t + W$$

where W is the dynamic waiting window.

The ride request is inserted into the active buffer along with its expiration time. During subsequent iterations of the batching algorithm, requests whose expiration time has passed are removed from the buffer and are no longer considered for matching.

This mechanism ensures that ride requests are only matched with temporally compatible candidates while maintaining bounded passenger waiting times.

3.5 Justification of Parameter Choices

The dynamic batching framework involves several parameters that may appear heuristic at first glance. In this section, we provide both empirical and theoretical justification for these parameter choices, ensuring that they are grounded in observed system behavior and not arbitrarily selected.

3.5.1 Choice of Base Waiting Time (3, 5, 7 minutes)

The base waiting time W_{base} determines the initial temporal window for batching before contextual adjustments are applied.

We evaluate three representative values: 3, 5, and 7 minutes. The corresponding results are shown in Table 4.2.

Base Wait	Total Rides	Batched Rides	Avg/Day	Batch %
3 min	1,657,148	56,472	4,033.7	3.41%
5 min	1,657,148	68,958	4,925.6	4.16%
7 min	1,657,148	93,092	6,649.4	5.62%

Table 3.1: Batching performance across base waiting times

While increasing the base wait time improves match rate. Specifically:

- Increasing from 3 to 5 minutes yields a +0.75% increase in batching.
- Increasing from 5 to 7 minutes yields a +1.45% increase.

However, this improvement comes at the cost of increased waiting time for all users. Notably, only a small fraction of users benefit from longer waiting windows, while the majority experience unnecessary delay.

Thus, the selected values (3, 5, 7) represent a controlled exploration of the trade-off between batching efficiency and user experience, rather than arbitrary choices.

3.5.2 Why Not Intermediate Values (e.g., 4 or 6 minutes)?

Intermediate values were not explicitly evaluated because the batching curve exhibits smooth, monotonic behavior with diminishing returns. The chosen values (3, 5, 7) sufficiently capture the trend:

- 3 minutes: lower bound (strict user experience constraint)
- 5 minutes: balanced operating point
- 7 minutes: upper practical limit before diminishing returns

Testing intermediate values would not significantly alter conclusions, as the system behavior is continuous and not sensitive to small perturbations.

3.5.3 Clipping Bounds (3 to 8 minutes)

The final waiting time is bounded as:

$$W = \min(8, \max(3, W_{raw}))$$

This clipping ensures service reliability and prevents extreme waiting times.

Upper Bound Justification (8 minutes):

Empirical analysis shows that:

- Average actual wait time $\approx$ 1.6–1.7 minutes
- Less than 0.02% of rides reach the maximum allowed wait

This implies that the upper bound is almost never active. Increasing the bound (e.g., to 10 minutes) does not improve match rates, indicating saturation of batching opportunities.

Thus, 8 minutes serves as a safe upper limit that preserves system flexibility without degrading user experience.

Lower Bound Justification (3 minutes):

A minimum wait time is necessary to accumulate potential matches. From a stochastic arrival perspective, assuming ride arrivals follow a Poisson process with rate λ, the expected waiting time for at least one additional request is:

$$\mathbb{E}[T] = \frac{1}{\lambda}$$

Empirically, this value lies between 1–2 minutes in high-demand zones. Therefore, a minimum window of 3 minutes ensures sufficient opportunity for batching while avoiding premature dispatch.

3.5.4 Distance Adjustment (1 minute per 5 km)

The distance adjustment is defined as:

$$F_{distance} = \min\left(\frac{\text{distance}}{5000}, 4\right)$$

This implies an increase of approximately 1 minute per 5 km.

Justification:

- Longer trips have higher tolerance for small delays.
- Detour cost scales sub-linearly with trip length.
- Empirically, average trip lengths and speeds (15–25 km/h) imply that 5 km corresponds to $\approx$ 12–20 minutes of travel time.

Thus, adding 1 minute per 5 km corresponds to a detour tolerance of roughly 5–8%, which is acceptable in ride-sharing systems.

3.5.5 Demand Density Thresholds

Demand thresholds (e.g., $D > 50$ or $D < 10$ requests per hour) are used to classify regions into high- and low-demand regimes for dynamic waiting time adjustment.

Theoretical Justification (Poisson Arrival Model):

Ride request arrivals in a localized spatial region can be reasonably approximated as a Poisson process with rate λ (requests per minute). Under this assumption, the probability of observing at least one additional request within a waiting window t is given by:

$$P(N(t) \geq 1) = 1 - e^{-\lambda t}$$

For the chosen thresholds:

- Low demand: $D < 10$ requests/hour $\Rightarrow \lambda < 0.17$ requests/min
- High demand: $D > 50$ requests/hour $\Rightarrow \lambda > 0.83$ requests/min

Evaluating the probability of at least one additional arrival within a typical waiting window of $t = 3$ minutes:

- Low demand: $P \approx 1 - e^{-0.5} \approx 39\%$
- High demand: $P \approx 1 - e^{-2.5} \approx 92\%$

Interpretation:

- In high-demand regions, the probability of finding a matching ride within a short time window is already very high ($> 90\%$). Hence, increasing the waiting time provides limited additional benefit, and the system reduces the waiting window.
- In low-demand regions, the probability of a match within the same window is significantly lower ($< 40\%$). Therefore, extending the waiting window improves the likelihood of successful batching.

3.5.6 Supply (ETA) Thresholds

Supply conditions are approximated using driver Estimated Time of Arrival (ETA), which reflects the time required for a driver to reach the pickup location.

- High supply: ETA < 180 seconds
- Low supply: ETA > 600 seconds

These thresholds are derived by interpreting ETA in terms of effective spatial distance using average urban travel speeds.

Assuming an average driver speed of approximately 20 km/h in urban conditions, we obtain:

- 180 seconds ($\approx$ 3 minutes) $\Rightarrow \approx$ 1 km distance
- 600 seconds ($\approx$ 10 minutes) $\Rightarrow \approx$ 3–3.5 km distance

Interpretation:

- When ETA $<$ 3 minutes ($\sim$1 km), a driver is already very close to the rider. In such cases, waiting for batching provides limited additional benefit, as a near-immediate pickup is feasible. Therefore, the waiting window is reduced.
- When ETA $>$ 10 minutes ($\sim$3 km+), the driver is relatively far from the rider. Users are already expected to wait for a longer duration, making it reasonable to slightly increase the batching window to improve matching probability.

This interpretation aligns with practical ride-hailing behavior: users perceive waiting time in relation to driver proximity rather than absolute time alone. A nearby driver implies immediacy, while a distant driver implicitly signals acceptable waiting tolerance.

3.5.7 Key Insight

Across all experiments, the most important observation is:

- Average actual wait time is significantly lower than allowed wait time
- Match rate remains stable across parameter variations

This indicates that batching is primarily limited by spatial and demand constraints rather than waiting time. Therefore, the chosen parameters provide a stable and robust operating regime without overfitting to specific values.

3.6 Spatio-Temporal Matching Engine

The batching engine processes ride requests sequentially in chronological order. Each incoming request attempts to match with compatible candidates that are currently waiting in the active buffer.

The algorithm maintains a pool of candidate rides that have not yet been matched and whose waiting window has not expired.

3.6.1 Matching Procedure

For each incoming ride request r_i, the following sequence of operations is performed:

1. Remove expired ride requests from the active buffer whose waiting window has elapsed.
2. Identify candidate rides whose pickup hexagons lie within the neighbourhood $N(h_p)$ of the new request.
3. For each candidate ride r_j, verify whether the drop-off hexagon lies within the neighbourhood of the new request's drop-off hexagon.
4. If both spatial conditions are satisfied, the two rides are grouped into a batch and assigned a common batch identifier.
5. If no compatible candidate is found, the ride request is inserted into the active buffer and waits for future matching opportunities.

Algorithm 1 Spatio-Temporal Ride Batching

```
for each incoming ride request r_i in chronological order do
    Remove expired rides from buffer
    for each candidate ride r_j in pickup neighborhood do
        if drop-off compatibility satisfied then
            Create batch (r_i, r_j)
            Remove r_j from buffer
            Continue
        end if
    end for
    Compute dynamic wait time W
    Insert r_i into buffer with expiry t_i + W
end for
```

3.7 Data Structures

Efficient data structures are required to support real-time ride matching while keeping computational overhead low. The batching algorithm relies on the following core data structures:

- **Active Buffer:** The active buffer stores ride requests that are currently waiting to be matched. Each ride remains in this buffer until either a compatible match is found or its waiting window expires.

- **Priority Queue (Min-Heap):** A priority queue is used to track the expiration time of each ride request. By always maintaining the earliest expiration time at the top of the heap, the algorithm can efficiently remove expired rides before processing new requests.
- **Spatial Index (Hash Map):** A hash map indexed by pickup hexagon stores references to active ride requests in each spatial cell. This allows the matching engine to quickly retrieve candidate rides from neighbouring pickup regions instead of searching the entire buffer.

3.8 Computational Complexity

Let n denote the total number of ride requests and B denote the maximum number of rides stored in the active buffer.

Each ride request is inserted into the heap at most once and removed at most once, resulting in heap operations with cost

$$O(n \log B)$$

Candidate searches are restricted to neighbouring pickup hexagons, which contain a constant number of cells. If k denotes the average number of candidate rides within these neighbourhoods, then candidate comparisons require $O(nk)$ operations.

Thus the overall runtime complexity can be approximated as

$$O(n \log B + nk)$$

In practice the waiting window bounds the buffer size and spatial partitioning limits neighbourhood density, making both B and k relatively small. Under typical operating conditions the algorithm therefore scales approximately linearly with the number of ride requests.

Worst-case behaviour can occur if many rides accumulate within the same spatial cell and remain active for long waiting windows. However, the combined spatial and temporal pruning mechanisms significantly reduce the likelihood of such scenarios in real ride-sharing systems.

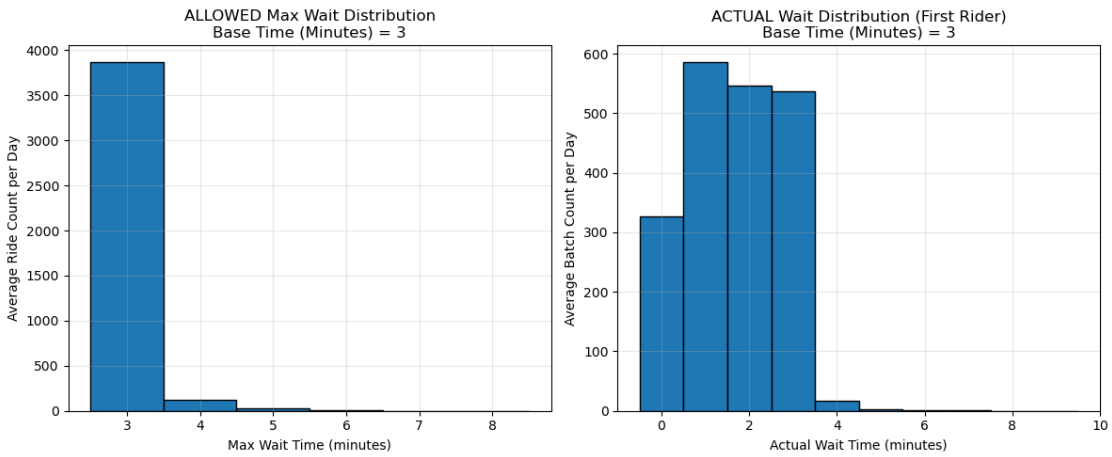


Figure 3.2: Comparison of Allowed Maximum Wait Distribution and Actual Wait Distribution for Base Time = 3 minutes.

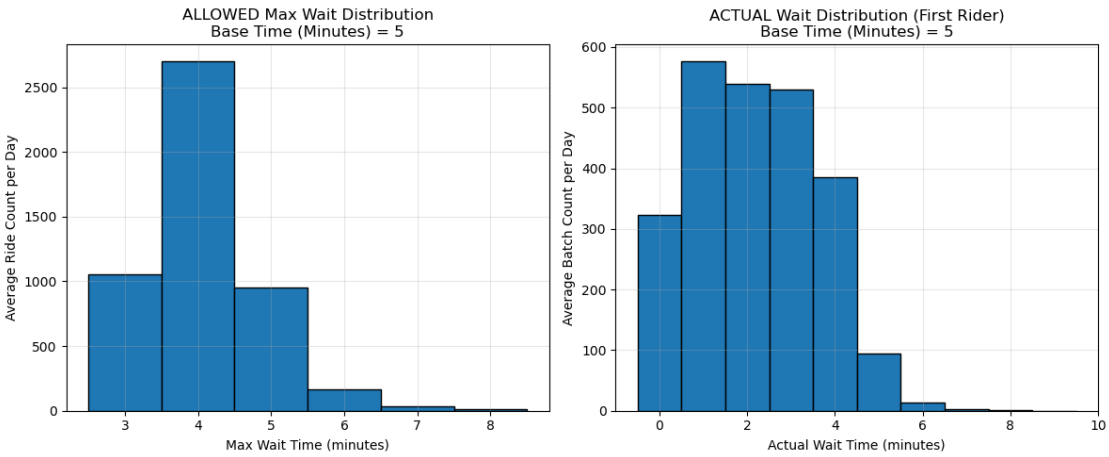


Figure 3.3: Comparison of Allowed Maximum Wait Distribution and Actual Wait Distribution for Base Time = 5 minutes.

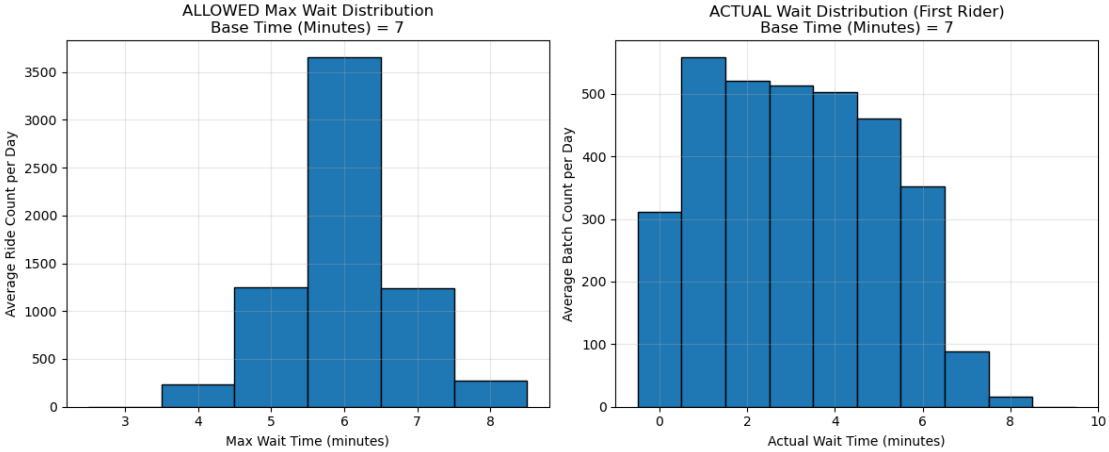


Figure 3.4: Comparison of Allowed Maximum Wait Distribution and Actual Wait Distribution for Base Time = 7 minutes.

Chapter 4

NY Pool

Ride-hailing platforms generate massive volumes of concurrent trip requests, and the ability to dynamically pool compatible rides assigning multiple passengers to a single vehicle offers significant reductions in operational cost, traffic congestion, and carbon emissions. Unlike static pooling (which matches requests before dispatch), dynamic or en-route pooling attempts to match a new incoming request with a vehicle that is already in transit, subject to constraints that bound the additional detour imposed on the existing passenger.

NY Pool operationalises this goal through the following high-level architecture:

1. **Trajectory Representation.** Every historical and ongoing ride is encoded as an ordered sequence of H3 hexagonal grid cells at resolution 9, forming a discrete spatial trajectory.
2. **Metadata Precomputation.** A preprocessed lookup table stores per-ride summary statistics start/end coordinates, journey hex sequences, and trip timestamps so that the streaming phase avoids repeated OSRM calls or recomputation.
3. **Sweep-Line Streaming.** GPS event logs are processed file-by-file in chronological order. A global sweep line advances through time, maintaining a live set of active rides and expiring those whose trip end time has passed.
4. **Hierarchical Candidate Filtering.** For each new pickup event, candidate pool partners are identified through a cascade of increasingly expensive filters: spatial proximity lookup → trajectory similarity scoring → road-network detour validation.
5. **Match Recording.** Confirmed poolable pairs are appended to an output list and persisted as a Parquet file for downstream evaluation.

The remainder of this chapter is organised as follows. Section 4.1 formally defines the dynamic pooling problem. Section 4.2 introduces the H3 hexagonal indexing scheme. Section 4.3 describes the core data structures. Section 4.4 formally defines each computational subroutine. Section 4.5 details the sweep-line streaming algorithm. Section 4.6 discusses computational complexity and memory characteristics. Section 4.7 discusses key design choices and limitations.

4.1 Problem Formulation

4.1.1 Ride Representation

Each ride request in the system is formally represented as a tuple:

$$r = \Big(p_r,\ d_r,\ t_r^s,\ t_r^e,\ T_r\Big), \tag{4.1}$$

where $p_r = (\phi_r^p, \lambda_r^p)$ denotes the pickup location, $d_r = (\phi_r^d, \lambda_r^d)$ the drop-off location, t_r^s and t_r^e the trip start and end timestamps, and $T_r = \langle h_1, h_2, \ldots, h_n \rangle$ the spatial trajectory encoded as an ordered sequence of H3 resolution-9 hexagonal cells traversed during the trip.

At any instant t during a trip, the ride occupies a current position $c_r(t) = (\phi_r(t), \lambda_r(t))$, which maps to a current hex cell $h_r(t) \in \mathcal{H}_9$.

4.1.2 Active Ride Set

Let $\mathcal{A}(t)$ denote the set of rides that are actively in transit at time t:

$$\mathcal{A}(t) = \Big\{ r : t_r^s \leq t \leq t_r^e \Big\}. \tag{4.2}$$

These are rides whose vehicles are currently on the road and potentially available to accommodate a second passenger.

4.1.3 The Dynamic Pooling Problem

Given an incoming pickup request B arriving at time t_B with pickup location p_B and destination d_B, the dynamic pooling problem seeks to identify an active ride $A \in \mathcal{A}(t_B)$ such that the combined trip in which the vehicle carrying passenger A detours to pick up and deliver passenger B satisfies the following constraints.

Constraint 1: Spatial Proximity. The current position of ride A at time t_B must lie within a bounded spatial neighbourhood of the pickup location of ride B:

$$d_{\text{hex}}\Big(h_A(t_B),\, h_B^{\text{start}}\Big) \leq k_{\text{prox}}, \tag{4.3}$$

where d_{hex} is the H3 grid distance and $k_{\text{prox}} = 2$ (corresponding to a spatial radius of approximately 350 m).

Constraint 2: Directional Compatibility. The net travel directions of rides A and B must be broadly aligned. Denoting the net bearings of rides A and B as θ_A and θ_B respectively, the pair must satisfy:

$$\Delta\theta(A, B) = \min\Big(|\theta_A - \theta_B|,\; 360^\circ - |\theta_A - \theta_B|\Big) < \delta, \tag{4.4}$$

with $\delta = 90^\circ$.

Constraint 3 Trajectory Overlap. The remaining trajectories of rides A and B must exhibit sufficient spatial overlap. Using the trajectory similarity score σ (defined formally in Section 4.4):

$$\sigma\Big(T_A^{\text{rem}},\, T_B^{\text{rem}}\Big) > \sigma_{\min}, \tag{4.5}$$

where $\sigma_{\min} = 0$ and T_A^{rem}, T_B^{rem} are the remaining hex trajectories from each ride's current position onwards. $\sigma_{\min} = 0$ means all rides which satisfied constraints 1 and 2 will pass this constraint as well, but we only choose the one with the max σ value among all candidates for next constraint.

Constraint 4: Bounded Detour. The additional road-network distance imposed on each passenger by the pooled route must not exceed a fraction τ of their respective direct (solo) trip distances:

$$\delta_A \leq \tau \quad \text{and} \quad \delta_B \leq \tau, \tag{4.6}$$

where δ_A and δ_B are the percentage detours for passengers A and B respectively, and $\tau = 30\%$.

4.1.4 Objective

Among all active rides satisfying Constraints (4.3)–(4.6), the system selects the candidate A^* that maximises trajectory similarity with ride B:

$$A^* = \arg\max_{A \in \mathcal{A}(t_B)} \sigma\Big(T_A^{\text{rem}}, T_B^{\text{rem}}\Big) \quad \text{subject to (4.3)–(4.6).} \tag{4.7}$$

If no feasible candidate exists, ride B proceeds as a solo trip after its assigned trip start time.

4.1.5 Distinction from Static Batching

It is important to note how this formulation differs from the static batching problem addressed by NY Share (Chapter 3). In NY Share, matching occurs before dispatch: two or more riders with nearly identical origins and destinations are grouped into a batch before any vehicle is assigned, guaranteeing zero detour. In NY Pool, matching occurs during transit: ride A is already en-route when ride B arrives, and the vehicle may physically deviate from its current path to accommodate the new passenger. This en-route nature introduces the detour constraint (Equation (4.6)) and requires real-time road-network routing queries that are absent from the static case. Consequently, the candidate filtering pipeline must be substantially more sophisticated, motivating the hierarchical architecture described in the subsequent sections.

4.2 Spatial Indexing using H3

4.2.1 Trajectory Encoding

Each trip is represented as a sequence of H3 cells visited along the driven route:

$$T = \langle h_1, h_2, \ldots, h_n \rangle, \quad h_i \in \mathcal{H}_9, \tag{4.8}$$

where $\mathcal{H}_9$ denotes the set of all valid resolution-9 H3 cells. This sequence is stored as part of the precomputed ride metadata.

4.2.2 Grid Disk Neighbourhoods

A key primitive used in both trajectory similarity scoring and proximity lookup is the k-disk neighbourhood of a cell:

$$D(h, k) = \Big\{ h' \in \mathcal{H}_9 : d_{\text{hex}}(h, h') \leq k \Big\}, \tag{4.9}$$

where d_{hex} is the grid distance (number of hex steps) between two cells. A k-disk centred on h contains $1 + 3k(k+1)$ cells; for $k = 2$ this yields 19 cells, and for $k = 1$, 7 cells (the cell itself plus its 6 immediate neighbours). This is exploited in the similarity metric to allow for small GPS deviations or route discretisation artefacts.

4.3 Core Data Structures

The streaming algorithm maintains six primary data structures in memory simultaneously. Their roles are summarised in Table 4.1 and described below.

4.3.1 Temporal Priority Queue

The priority queue stores tuples of the form (t_{end}, *seq*, *rid*), sorted in ascending order of trip end time (nanoseconds since epoch). The sequence counter *seq* breaks ties deterministically. When the sweep line advances to timestamp t, all entries with $t_{\text{end}} \leq t$ are popped and their corresponding rides are evicted from the active ride registry and the spatial inverted index.

Proposition 4.3.1 (Eviction Complexity)**.** Evicting m expired rides from the priority queue costs $O(m \log M)$ amortised time, where M is the maximum number of simultaneously active rides.

Table 4.1: Global data structures maintained during the sweep-line streaming phase.

Structure	**Type**	**Purpose**
Temporal priority queue	Min-heap	Ordered by trip end time; enables efficient ride expiry.
Active ride registry	Dictionary	Stores live ride state: GPS position, hex cell, trajectory.
Spatial inverted index	Dictionary of sets	Maps each occupied H3 cell to the set of active rides within it.
Matched ride set	Set	Rides already assigned to a pool pair; prevents double-matching.
Attempt throttle map	Dictionary	Records last match attempt time; enforces a 15 s cooldown.

4.3.2 Spatial Inverted Index

This structure implements a spatial lookup table, mapping each H3 cell to the set of rides whose current position falls within it:

$$\mathcal{I}[h] = \Big\{\, rid : \text{current hex of ride } rid = h \,\Big\}. \tag{4.10}$$

Candidate pooling partners for a new ride B arriving at hex h_B are retrieved by querying all cells in $D(h_B, 2)$ the 2-disk neighbourhood in $O(19)$ dictionary lookups, each $O(1)$ on average. This replaces a naïve $O(M)$ linear scan over all active rides.

4.3.3 Ride Metadata Cache

This lookup table is preloaded from a serialised file and holds, for each ride identifier, the following fields:

- Trip start and end timestamps bounding the trip window.
- Origin and destination coordinates (latitude, longitude).
- The precomputed H3 resolution-9 trajectory sequence.

Loading from a serialised file avoids re-deriving these summaries during the performance-critical streaming loop.

4.4 Computational Subroutines

4.4.1 Bearing Computation

The bearing subroutine computes the initial forward azimuth (in degrees, $[0, 360)$) from point (ϕ_1, λ_1) to point (ϕ_2, λ_2) using the spherical bearing formula:

$$\Delta\lambda = \lambda_2 - \lambda_1, \tag{4.11}$$

$$x = \sin(\Delta\lambda) \cdot \cos(\phi_2), \tag{4.12}$$

$$y = \cos(\phi_1) \cdot \sin(\phi_2) - \sin(\phi_1) \cdot \cos(\phi_2) \cdot \cos(\Delta\lambda), \tag{4.13}$$

$$\theta = \left(\operatorname{atan2}(x, y) \cdot \tfrac{180}{\pi}\right) \bmod 360, \tag{4.14}$$

where all trigonometric arguments are in radians. The result θ gives the compass bearing (in degrees clockwise from North) of the vector joining the two geographic points.

4.4.2 Directional Compatibility

Definition 4.4.1 (Directional Compatibility)**.** Two paths $P_A = \langle a_{curr}, a_{end} \rangle$ and $P_B = \langle b_{start}, b_{end} \rangle$ are *directionally compatible* with threshold δ if:

$$\Delta\theta(P_A, P_B) = \min\Big(|\theta_A - \theta_B|,\ 360 - |\theta_A - \theta_B|\Big) < \delta, \tag{4.15}$$

where θ_A and θ_B are the bearings of each path from its start to its end point.

The system uses $\delta = 90°$, meaning rides travelling in roughly perpendicular or more aligned directions are considered compatible. Rides travelling in opposite directions (e.g., northbound vs. southbound) are immediately rejected without further computation. This early rejection substantially reduces the number of expensive trajectory similarity computations.

4.4.3 Trajectory Similarity Score

This is the core spatial compatibility metric. Given two H3 trajectory sequences T_A and T_B (the remaining portions from each ride's current position onwards), the similarity is computed as follows.

Step 1: Directional Gate. If the two trajectories fail the directional compatibility check, immediately return a score of 0.0.

Step 2: Expanded Cell Sets. For each trajectory, compute the union of k-disk neighbourhoods over all its constituent cells:

$$S_A = \bigcup_{h \in T_A} D(h, k), \qquad S_B = \bigcup_{h \in T_B} D(h, k). \tag{4.16}$$

With $k = 2$, each cell expands to a 19-cell disk, effectively thickening each trajectory by $\sim 350\,\text{m}$ on each side. This tolerates small route deviations and hex-boundary artefacts.

Step 3: Coverage Ratios. Compute the fraction of T_A's cells that fall within the expanded neighbourhood of T_B, and vice versa:

$$c_A = \frac{|\{h \in T_A : h \in S_B\}|}{|T_A|}, \qquad c_B = \frac{|\{h \in T_B : h \in S_A\}|}{|T_B|}. \tag{4.17}$$

Step 4: Conservative Score. The similarity score is defined as the minimum of the two coverage ratios:

$$\sigma(T_A, T_B) = \min(c_A, c_B) \in [0, 1]. \tag{4.18}$$

Taking the minimum (rather than the mean or harmonic mean) enforces a conservative similarity criterion: both rides must have large fractions of their remaining routes overlapping with the other, preventing asymmetric matches where one short ride is entirely contained within a long ride's trajectory.

Remark. The use of k-disk expansion transforms the similarity metric from a set intersection on exact hex cells to a proximity-aware overlap. This is essential in practice because H3 trajectories derived from GPS traces may have small positional offsets even for rides traversing the same physical road segment.

4.4.4 Road-Network Detour Validation

Once a candidate pair (A, B) passes the trajectory similarity threshold, the system invokes a locally hosted OSRM (Open Source Routing Machine) instance to obtain precise road-network driving distances for the proposed combined trip. This is the most computationally expensive filter and is only invoked for high-confidence candidates.

Route Order Enumeration

Given the current positions of ride A (A_{curr}) and ride B (B_{curr}), and their respective destinations (A_{end}, B_{end}), two possible pooled route orderings are evaluated:

$$\text{Order 1 (ABAB):} \quad A_{\text{curr}} \to B_{\text{curr}} \to A_{\text{end}} \to B_{\text{end}}, \tag{4.19}$$

$$\text{Order 2 (ABBA):} \quad A_{\text{curr}} \to B_{\text{curr}} \to B_{\text{end}} \to A_{\text{end}}. \tag{4.20}$$

Each ordering corresponds to a different drop-off sequence: in Order 1, passenger A is dropped first; in Order 2, passenger B is dropped first. The system selects the ordering with the shorter total distance:

$$\text{selected order} = \arg\min\Big\{ \text{dist(Order 1)},\ \text{dist(Order 2)} \Big\}. \tag{4.21}$$

Routing Queries

Three route queries are issued to the OSRM server:

1. Route distance legs for Order 1 waypoints $[A_{\text{curr}}, B_{\text{curr}}, A_{\text{end}}, B_{\text{end}}]$.
2. Route distance legs for Order 2 waypoints $[A_{\text{curr}}, B_{\text{curr}}, B_{\text{end}}, A_{\text{end}}]$.
3. Direct (solo) route distance for ride A: $[A_{\text{curr}}, A_{\text{end}}]$.

The direct distance for ride B is extracted from the second query (the segment $B_{\text{curr}} \to B_{\text{end}}$ in Order 2, which constitutes a direct leg). All requests are configured to return only per-leg distances without full route polylines.

Detour Computation

For the selected ordering, the percentage detour suffered by each rider is computed as follows.

Order 1 (ABAB) selected:

$$\delta_A = \frac{(\ell_0 + \ell_1) - d_A^{\text{direct}}}{d_A^{\text{direct}}} \times 100\%, \tag{4.22}$$

$$\delta_B = \frac{(\ell_1 + \ell_2) - d_B^{\text{direct}}}{d_B^{\text{direct}}} \times 100\%, \tag{4.23}$$

where $\ell_0 = d(A_{\text{curr}}, B_{\text{curr}})$, $\ell_1 = d(B_{\text{curr}}, A_{\text{end}})$, $\ell_2 = d(A_{\text{end}}, B_{\text{end}})$ are the individual route legs.

Order 2 (ABBA) selected:

$$\delta_A = \frac{(\ell_0 + \ell_1 + \ell_2) - d_A^{\text{direct}}}{d_A^{\text{direct}}} \times 100\%, \tag{4.24}$$

$$\delta_B = 0\%. \tag{4.25}$$

In Order 2, passenger B is dropped first, so their ride is never extended beyond the direct segment to their destination ($\delta_B = 0$). Passenger A absorbs the entire detour from picking up and then dropping off B.

Feasibility Decision

The pair is declared batchable if and only if both detours are within the allowed threshold $\tau = 30\%$:

$$\text{batchable} \iff \delta_A \leq \tau \ \wedge \ \delta_B \leq \tau. \tag{4.26}$$

The subroutine returns the feasibility flag along with the direct distances, selected ordering, total pooled distance, individual detour percentages, pickup leg distance, and inter-destination distance.

4.5 The Sweep-Line Streaming Algorithm

4.5.1 File Discovery and Loading Strategy

The system operates over a directory of Apache Parquet files, each containing a chronological slice of GPS event records. Files are sorted lexicographically (which, given the naming convention, corresponds to chronological order) and processed sequentially. Only five columns are loaded per file: ride identifier, timestamp, latitude, longitude, and ride status.

This selective loading drastically reduces memory pressure compared to loading full-width Parquet files. After each file is processed, the loaded data is explicitly released to free memory before the next file is loaded.

Each loaded chunk undergoes the following preprocessing:

1. Coerce latitude and longitude to numeric values, dropping rows with invalid coordinates.
2. Parse timestamps into a standard datetime representation.
3. Compute the H3 resolution-9 cell for every GPS record.
4. Encode ride status as a categorical type for memory efficiency.
5. Sort all records chronologically.

4.5.2 The Sweep-Line Event Loop

The algorithm processes each GPS event record (t_i, rid_i, h_i, s_i) in ascending chronological order, where t_i is the timestamp, h_i is the current hex cell, and $s_i \in \{\texttt{PICKUP}, \texttt{ON_RIDE}, \ldots\}$ is the ride status.

4.5.3 Ride Lifecycle and Status Handling

The system distinguishes between two ride statuses that trigger distinct logic branches.

On_ride status (new encounter). When a ride is first observed in the en-route status, it is eligible to become a pool partner (ride A). It is inserted into the active ride registry with its current GPS coordinates, destination, and hex trajectory. Its end-time is pushed onto the temporal priority queue to ensure eventual eviction, and it is registered in the spatial inverted index under its current H3 cell.

Pickup status (new encounter, not already matched, not recently attempted). When a ride first appears in the pickup status, it represents an incoming request (ride B) seeking a pool partner from the active set. The full matching pipeline is triggered (see Section 4.5.6). If matching fails, the ride's timestamp is recorded in the attempt throttle map to suppress re-attempts for 15 seconds.

Subsequent events (ride already active). For any status, if the ride is already known, only its current position is updated: the old hex cell is removed from the spatial inverted index, the new hex cell is added, and the registry entry is refreshed with the latest GPS fix.

4.5.4 Expiry and Eviction

At the start of processing each event at time t, the priority queue is polled to evict all rides whose recorded end time has passed. The eviction procedure proceeds as follows:

1. Pop the earliest-expiring entry from the priority queue.
2. If the ride is still present in the active registry, remove its hex cell entry from the spatial inverted index.
3. If the hex cell entry in the index becomes empty, delete it entirely to prevent unbounded growth over long runs.
4. Remove the ride from the active registry and increment the eviction counter.

This loop repeats until no more expired entries remain at the head of the queue.

4.5.5 Matching Attempt Throttling

The attempt throttle map records the nanosecond timestamp of the most recent matching attempt for each ride. A ride B is considered recently attempted if:

$$t_{\text{current}} - t_{\text{last_attempt}}[rid] \;\leq\; 15 \times 10^9\,\text{ns} \;=\; 15\,\text{seconds}. \tag{4.27}$$

This prevents the system from repeatedly issuing expensive routing queries for the same ride within a short window if it is temporarily unmatched.

4.5.6 Hierarchical Filtering Pipeline

The matching pipeline for a new pickup ride B proceeds through four hierarchical filters, each substantially reducing the candidate set before the next (more expensive) filter is applied. Figure 4.1 illustrates the cascade.

Algorithm 2 SweepLinePooling($\mathcal{F}$, $\mathcal{M}$, δ, k, τ_{cd})

Require: Event files $\mathcal{F}$, metadata $\mathcal{M}$, detour threshold δ, disk radius k, cooldown τ_{cd}
Ensure: Set of matched ride pairs $\mathcal{P}$

Initialise $\mathcal{A} \leftarrow \emptyset,\ \mathcal{H} \leftarrow \emptyset,\ \mathcal{X} \leftarrow \emptyset,\ \mathcal{B} \leftarrow \emptyset,\ \mathcal{T} \leftarrow \emptyset,\ \mathcal{P} \leftarrow \emptyset$
for each file $f \in \mathcal{F}$ in chronological order **do**
    Load events from f; sort by timestamp t
    for each event $e = (\text{rid}, t, \text{lat}, \text{lon}, \text{status})$ **do**
        hex $\leftarrow$ H3Cell(lat, lon, res = 9)
        $m \leftarrow \mathcal{M}[\text{rid}]$

— Step 1: Evict expired rides —

while $\mathcal{H} \neq \emptyset$ **and** $\mathcal{H}$.peek().end_time $\leq t$ **do**

  $(_, \text{rid}') \leftarrow \mathcal{H}$.pop()

  Remove rid' from $\mathcal{A}$ and $\mathcal{X}$

end while

— Step 2: Update existing ride position —

if rid $\in \mathcal{A}$ **then**

  Update $\mathcal{X}$: move rid from old hex to hex

  $\mathcal{A}[\text{rid}]$.hex $\leftarrow$ hex; $\mathcal{A}[\text{rid}]$.lat, $\mathcal{A}[\text{rid}]$.lon $\leftarrow$ lat, lon

end if

— Step 3: Pickup matching —

if rid $\notin \mathcal{A}$ **and** rid $\notin \mathcal{B}$ **and** status = `ON_PICKUP` **then**

  if rid $\in \mathcal{T}$ **and** $t - \mathcal{T}[\text{rid}] \leq \tau_{\text{cd}}$ **then**

    continue ▷ Within cooldown window; skip

  end if

  $\mathcal{T}[\text{rid}] \leftarrow t$

  $\text{hex}_B \leftarrow$ H3Cell(m.start_lat, m.start_lon, 9)

  $N \leftarrow \bigcup_{h \in \text{GridDisk}(\text{hex}_B, 2)} \mathcal{X}[h]$ ▷ Neighbour rides within 2-ring disk

  $H_B^{\text{rem}} \leftarrow m$.hexes[idx($m$.hexes, hex_B) :]

  $s^* \leftarrow 0$; $A^* \leftarrow$ null

  for each $\text{rid}_A \in N \setminus \mathcal{B}$ **do**

    $a \leftarrow \mathcal{A}[\text{rid}_A]$

    $H_A^{\text{rem}} \leftarrow a$.hexes[idx($a$.hexes, a.hex) :]

    $C_A \leftarrow$ [(a.lat, a.lon), (a.end_lat, a.end_lon)]

    $C_B \leftarrow$ [(m.start_lat, m.start_lon), (m.end_lat, m.end_lon)]

    $s \leftarrow$ PathSimilarity(H_A^{rem}, H_B^{rem}, C_A, C_B, k)

    if $s > s^*$ **then**

      $s^* \leftarrow s$; $A^* \leftarrow \text{rid}_A$

    end if

  end for

  if $s^* > 0$ **and** $A^* \neq$ null **then**

    (ok, path, d_A, d_B, ϵ_A, ϵ_B) $\leftarrow$ DetourCheck(A^*_{curr}, B_{curr}, A^*_{end}, B_{end}, δ)

    if ok **then**

      $\mathcal{B} \leftarrow \mathcal{B} \cup \{A^*, \text{rid}\}$

      $\mathcal{P}$.append({A^*, rid, s^*, d_A, d_B, ϵ_A, ϵ_B, path, t})

    end if

  end if

end if

— Step 4: Register new on-ride trips —

if rid $\notin \mathcal{A}$ **and** status = `ON_RIDE` **then**

  Add rid to $\mathcal{A}$ with full metadata and current position

  $\mathcal{X}$[hex].add(rid)

  $\mathcal{H}$.push((m.end_time, rid))

end if

end for

Free file memory

end for

return $\mathcal{P}$

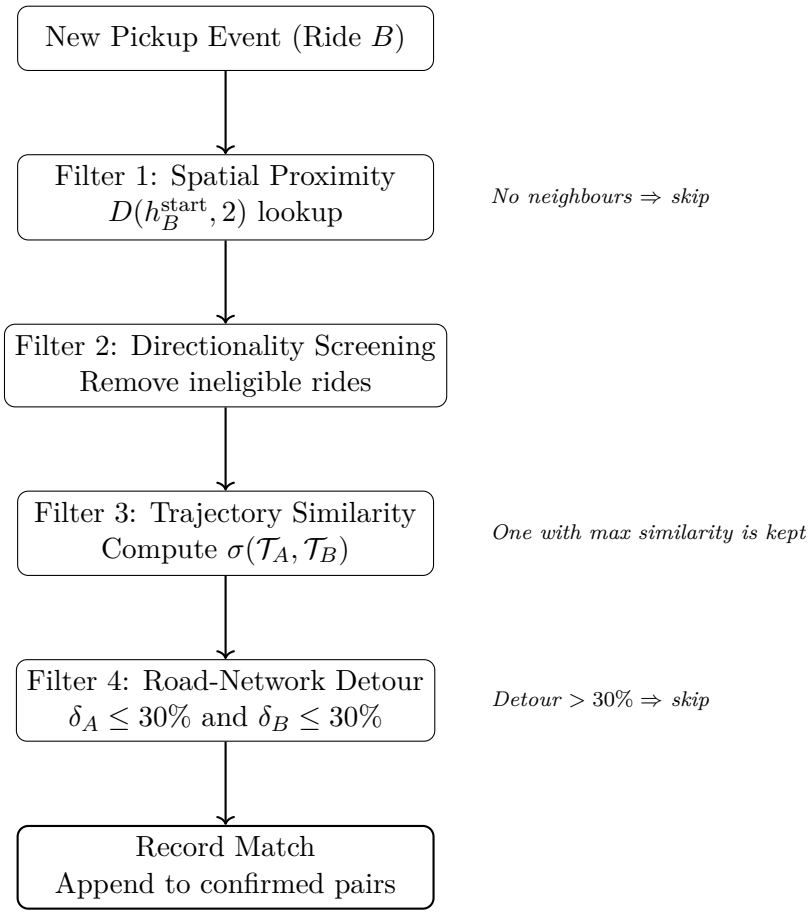


Figure 4.1: Hierarchical filtering pipeline for dynamic pooling candidate selection.

Total Rides	Batched Rides	Batch %
138,654	37,554	27.08%

Table 4.2: Batching performance of NY pool on 2nd Aug

4.6 Computational Complexity and Memory Analysis

4.6.1 Time Complexity

Let N be the total number of GPS event records across all files, M the peak number of simultaneously active rides, and K the average number of candidates retrieved per hex-disk lookup.

The dominant cost per pickup event is the trajectory similarity computation, which is $O(L \cdot K)$ where L is the average trajectory length in hex cells. The routing validation is invoked only for the single best candidate, bounding it at 3 network calls per pickup regardless of K.

Overall processing complexity is approximately:

$$O\Big(N \cdot (\log M + K \cdot L + \Omega_{\text{routing}})\Big), \tag{4.28}$$

where Ω_{routing} is the constant cost of 3 local HTTP requests.

4.6.2 Memory Complexity

The design is explicitly engineered for low memory overhead:

Table 4.3: Per-event time complexity of key operations.

Operation	**Complexity**	**Notes**
Hex cell computation	$O(1)$	Fixed-cost H3 library call
Ride expiry (amortised)	$O(\log M)$	Each ride evicted once
Spatial candidate lookup	$O(19 \cdot \bar{K})$	19 cells in 2-disk, $\bar{K}$ rides per cell
Directional compatibility check	$O(1)$	Constant arithmetic
Trajectory similarity (per cand.)	$O(\lvert T_A \rvert \cdot 19 + \lvert T_B \rvert \cdot 19)$	Set construction and lookup
Detour validation	$O(1)_{\text{network}}$	3 HTTP round-trips to local server

- **Input files:** Loaded one at a time; O(file size) at any point.
- **Active ride registry:** $O(M)$ entries, each storing $\sim$10 scalar fields and a hex list.
- **Spatial inverted index:** $O(M)$ total entries across all hex cells (each ride occupies exactly one cell).
- **Temporal priority queue:** $O(M)$ tuples.
- **Confirmed match list:** $O(P)$ where P is the number of matched pairs (typically $P \ll N$).

The ride metadata cache is the largest persistent object, consuming memory proportional to the number of unique rides in the dataset. This is a deliberate trade-off: by preloading metadata, the streaming loop avoids all disk I/O except sequential file reading.

4.7 Design Choices and Limitations

4.7.1 Key Design Decisions

1. **One-pass greedy matching.** Each ride is matched at most once (both rides are immediately marked as batched upon a successful match). This greedy strategy trades global optimality for computational tractability.
2. **Local routing server.** Using a locally deployed OSRM instance avoids external API rate limits and latency variability, making the throughput of routing queries predictable and high.
3. **Conservative detour threshold of 30%.** A 30% detour cap is a common industry standard for ride pooling, balancing passenger experience against batching yield.
4. $k = 2$ **disk expansion.** The choice of $k = 2$ was made to tolerate GPS positioning noise and hex-grid boundary artefacts without being so permissive as to match rides on parallel but non-overlapping corridors.

4.7.2 Limitations and Potential Improvements

1. **Greedy matching.** The greedy one-shot matching strategy may not maximise overall pooling yield. A global matching formulation (e.g., maximum weight bipartite matching over all active rides) would yield higher pool rates at greater computational cost.

2. **No capacity constraints.** The system assumes a two-passenger pool capacity. Extensions to $n > 2$ passengers would require a more general multi-party matching framework.

3. **Static metadata.** The ride metadata cache is built offline. Dynamic routing changes or mid-trip diversions are not reflected in the precomputed trajectories.

Chapter 5

Analysis of NY Share and NY Pool

5.1 Analysis of NY Share

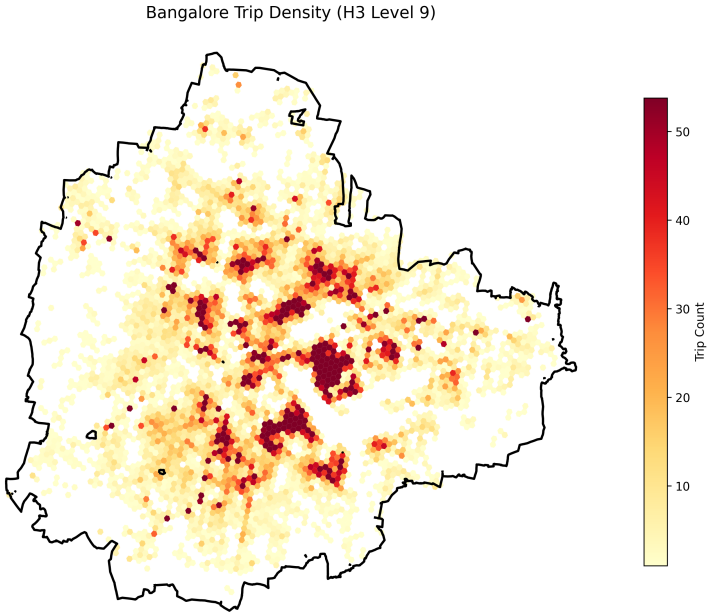


Figure 5.1: Density of batched rides for 14-days.

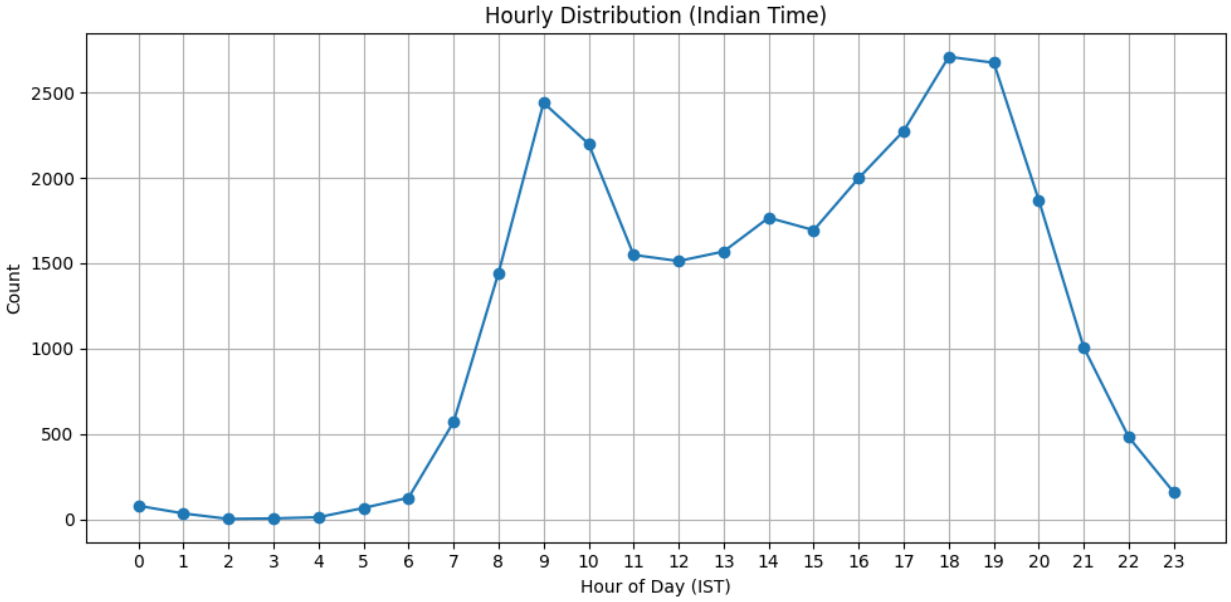


Figure 5.2: Hourly distribution of batched rides for 14-days.

The spatial distribution of batched rides across Bangalore over the 14-day period reveals a clear concentration of demand in the central and southern parts of the city, as shown in Figure 5.1. High-density clusters are prominent around the core urban areas and major commercial corridors, while demand diminishes significantly toward the city's periphery. This pattern

suggests that ride-batching opportunities are largely confined to well-connected, high-activity zones, making these areas the most suitable for deploying pooling-based algorithms.

Figure 5.2 illustrates the hourly distribution of batched rides over a 14-day period, revealing a distinct bimodal pattern characteristic of urban commuting cycles. Demand remains negligible during the early morning hours (0:00 to 5:00) before experiencing a sharp incline starting at 7:00, reaching a morning peak at 9:00 with approximately 2,450 rides. Following a mid-day plateau where ride counts hover between 1,500 and 1,800, a second, more sustained surge begins in the late afternoon. This evening peak occurs between 18:00 and 19:00, reaching the highest volume in the dataset at over 2,700 rides. The sharp decline after 20:00 suggests a rapid wind-down of activities as the day ends. This distribution highlights that the batched ride system is most heavily utilized during standard office "log-in" and "log-out" windows, indicating its primary role in supporting daily workforce transit.

5.2 Analysis of NY Pool

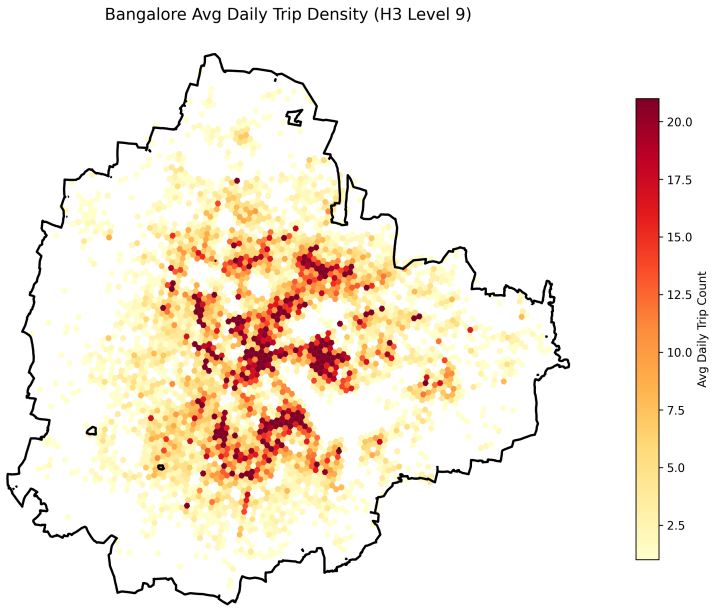


Figure 5.3: Density of batched rides for August 2, 2025

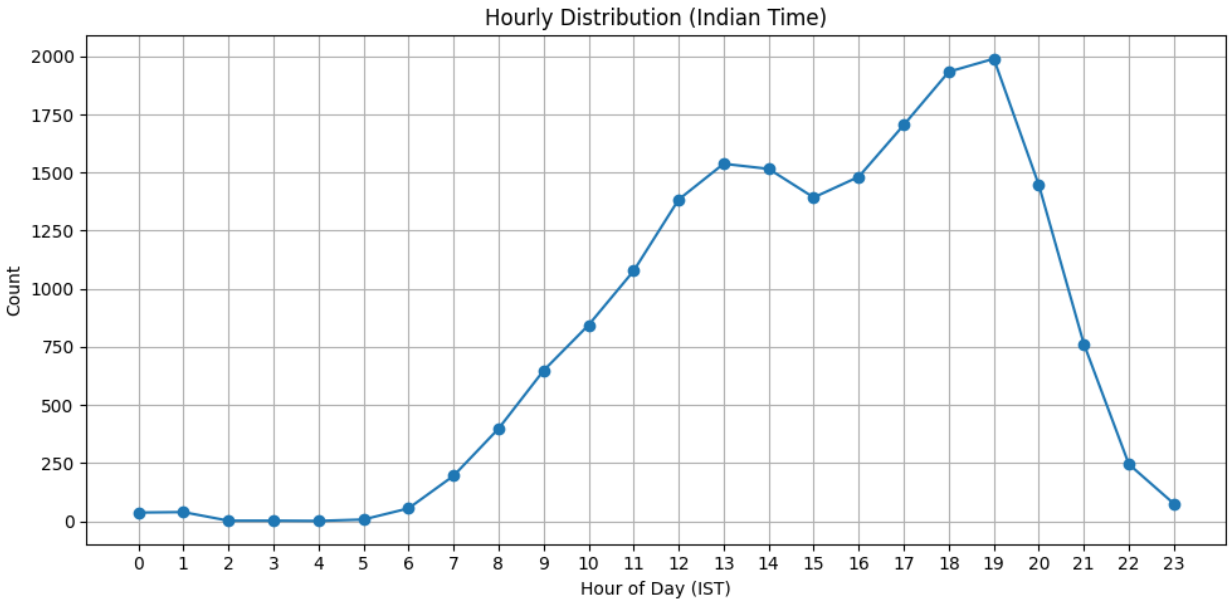


Figure 5.4: Hourly distribution of batched rides for August 2, 2025

The spatial distribution of average daily trips across Bangalore for August 2, 2025, reveals a clear concentration of demand in the central and south-eastern parts of the city. As shown in Figure 5.3, high-density clusters—indicating areas with over 15 to 20 average daily trips per H3 hexagonal cell—are prominent around the core urban areas and major commercial corridors. Conversely, the demand diminishes significantly toward the city's periphery, tapering down to fewer than 5 trips per day. This pattern suggests that ride-batching opportunities are heavily confined to well-connected, high-activity zones, making these central areas the most suitable for deploying pooling-based algorithms.

Figure 5.4 illustrates the hourly distribution of these rides, revealing a distinct temporal pattern characterized by a steady mid-day rise and a dominant evening peak. Demand

remains negligible during the early morning hours (0:00 to 5:00) before experiencing a gradual incline starting at 7:00. Rather than a sharp morning commuter peak, the volume climbs consistently through the morning to reach an initial plateau of approximately 1,500 rides between 13:00 and 14:00. Following a brief stabilization, a second, more sustained surge begins in the late afternoon. This evening peak reaches its maximum volume at 19:00 with nearly 2,000 recorded rides. The sharp decline after 20:00 suggests a rapid wind-down of activities as the day ends. This distribution highlights that the batched ride system is most heavily utilized during the afternoon and evening hours, indicating its primary role in supporting return commutes and late-day urban transit.

5.3 VKT Analysis (Vehicle Kilometers Traveled)

One of the most critical metrics for evaluating the environmental and infrastructural impact of a ride-sharing system is the reduction in Vehicle Kilometers Traveled (VKT). VKT reduction directly correlates to decreased emissions, lower fuel consumption, and an overall reduction in urban congestion.

5.3.1 VKT—NY Share

Figure 5.5 illustrates the percentage of distance saved through the batching approach for the NY Share dataset. The total direct distance serves as the baseline at 100%, while the distance after batching accounts for approximately 60.07% of the total, resulting in a saved distance of 39.93%. This demonstrates that the proposed batching mechanism significantly reduces the travel distance by nearly 40%, highlighting its effectiveness in optimizing route planning.

The environmental impact of this optimization is further quantified in Table 5.1, which presents the emission summary across key pollutants. The direct emissions for CO_2, NO_x, CO, and PM were recorded at 31474.66 kg, 41.97 kg, 419.66 kg, and 2.10 kg, respectively. After applying the batching strategy, these values were reduced to 18906.68 kg, 25.21 kg, 252.09 kg, and 1.26 kg, yielding saved emissions of 12567.98 kg, 16.76 kg, 167.57 kg, and 0.84 kg for each pollutant. These results clearly indicate that the batching approach not only improves distance efficiency but also contributes substantially to reducing vehicular emissions, making it a promising solution from both an operational and environmental sustainability perspective.

Table 5.1: Emission Summary (in kg) for NY Share

Pollutant	**Direct Emission**	**After Batching**	**Saved Emission**
CO_2	31474.6583	18906.6778	12567.9806
NO_x	41.9662	25.2089	16.7573
CO	419.6621	252.0890	167.5731
PM	2.0983	1.2604	0.8379

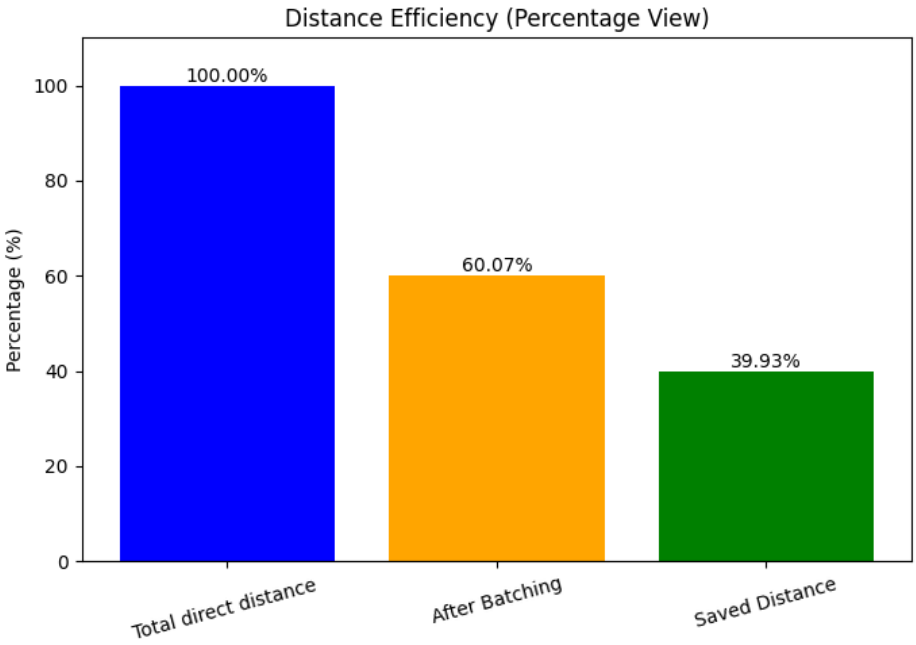


Figure 5.5: Percentage of distance saved for NY share

5.3.2 VKT—NY Pool

Figure 5.6 illustrates the percentage of distance saved through the dynamic en-route matching approach for the NY Pool dataset. Similar to the static batching evaluation, the total direct distance serves as the baseline at 100%. After applying the dynamic pooling algorithm, the total traveled distance accounts for 84.16%, resulting in a saved distance of 15.84%. We have made this plot on Aug 2, 2025 data.

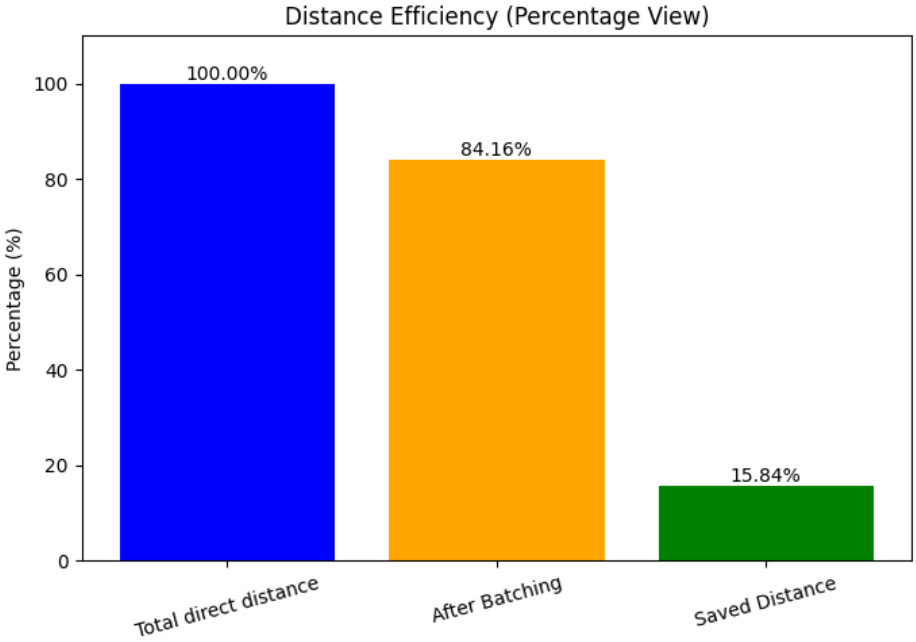


Figure 5.6: Percentage of distance saved for NY Pool

While the distance reduction achieved by NY Pool (15.84%) is more modest compared to the nearly 40% reduction observed in NY Share, this is an expected outcome of the system's architecture. Dynamic pooling operates under much stricter en-route constraints than static

batching. Specifically, the requirement to match a new request with a vehicle already in transit—bounded by a strict 30% maximum detour limit and the necessity for trajectory overlap —naturally restricts the poolable candidate set. Despite these constraints, saving nearly 16% of the total vehicle kilometers traveled represents a highly significant operational efficiency gain for active fleets.

The environmental impact of this optimized routing is quantified in Table 5.2, which presents the emission summary for key pollutants under the NY Pool model.

Table 5.2: Emission Summary (in kg) for NY Pool

Pollutant	**Direct Emission**	**After Batching**	**Saved Emission**
CO_2	37305.2352	31813.6190	5491.6162
NO_x	49.7403	42.4182	7.3222
CO	497.4031	424.1816	73.2215
PM	2.4870	2.1209	0.3661

Chapter 6

Conclusion

This report investigated the potential of advanced shared mobility algorithms specifically NY Share and NY Pool to mitigate escalating traffic congestion and environmental degradation in urban centers like Bengaluru. By leveraging the H3 hexagonal spatial indexing framework for trajectory representation and proximity mapping, both models demonstrated substantial operational efficiencies and reductions in Vehicle Kilometers Traveled (VKT).

NY Share, designed as a batching system, successfully groups spatially and temporally compatible ride requests prior to dispatch. By utilizing a dynamic waiting time mechanism that adapts to trip distance, time of day, and local neighborhood demand, the algorithm achieved a nearly 40% reduction in VKT. This optimization translated into profound environmental benefits, yielding a savings of over 12,567 kg in CO2 emissions and proportional reductions in particulate matter over the evaluated period.

Conversely, NY Pool addressed the more complex operational challenge of dynamic, en-route matching, pairing new incoming requests with vehicles already in transit. Employing a computationally efficient sweep line streaming algorithm and a rigorous hierarchical filtering pipeline incorporating trajectory similarity scoring and a strict 30% detour limit NY Pool secured a 15.84% reduction in VKT. While these savings are naturally more modest than those of static batching due to the strict constraints of en-route routing, they represent a highly significant efficiency gain for active vehicle fleets.

Ultimately, the analysis establishes that both NY Share and NY Pool are viable, scalable technological interventions capable of safely maximizing vehicle utilization. As Indian cities continue to grapple with rapid vehicle proliferation and the spatial limits of road infrastructure, these shared mobility frameworks offer a necessary and impactful complement to public transit expansion.

Bibliography

[1] Prottoy Akbar, Victor Couture, Gilles Duranton, and Adam Storeygard. *Mobility and Congestion in Urban India.* American Economic Review, 113(4): 1083–1111, 2023. https://www.aeaweb.org/articles?id=10.1257/aer.20181662

[2] Naman, Shah. *In Race to Build Metro Trains, How Indian Cities Ignored Larger Mobility Issue.* The Quint, 2024. https://www.thequint.com/videos/public-transport-problem-in-indian-cities-traffic-metro-trains-buses

[3] Shravani Sharma. *Recognising Shared Mobility as a Sustainable Form of Mobility in Urban Areas.* WRI India, 2021. https://wri-india.org/blogs/recognising-shared-mobility-sustainable-form-mobility-urban-areas

[4] Wang Chen, Guan Huang, and Jintao Ke. *Development Dilemma of Ride-Sharing: Revenue or Social Welfare?* arXiv preprint arXiv:2412.08801, 2024. https://arxiv.org/abs/2412.08801

[5] Prateek Aryyan. *How Uber Pool Decides Who Rides With Whom.* https://medium.com/@prateekaryyan/the-10-million-algorithm-how-uber-pool-decides-who-rides-with-whom-fe2f8135d376

[6] Yuval Emek, Shay Kutten, and Roger Wattenhofer. *Online Matching: Haste Makes Waste!* In *Proceedings of the 48th Annual ACM Symposium on Theory of Computing (STOC)*, pages 333–344, 2016. https://doi.org/10.1145/2897518.2897557

[7] Niels Agatz, Alan Erera, Martin Savelsbergh, and Xing Wang. *Optimization for Dynamic Ride-Sharing: A Review.* European Journal of Operational Research, 223(2): 295–303, 2012. https://doi.org/10.1016/j.ejor.2012.05.028

[8] Siddhartha Banerjee, Daniel Freund, and Thodoris Lykouris. *Pricing and Optimization in Shared Vehicle Systems: An Approximation Framework.* Operations Research, 70(3): 1783–1805, 2022. https://doi.org/10.1287/opre.2021.2165

[9] Javier Alonso-Mora, Samitha Samaranayake, Alex Wallar, Emilio Frazzoli, and Daniela Rus. *On-Demand High-Capacity Ride-Sharing via Dynamic Trip–Vehicle Assignment.* Proceedings of the National Academy of Sciences, 114(3): 462–467, 2017. https://doi.org/10.1073/pnas.1611675114

[10] Paolo Santi, Giovanni Resta, Michael Szell, Stanislav Sobolevsky, Steven H. Strogatz, and Carlo Ratti. *Quantifying the Benefits of Vehicle Pooling with Shareability Networks.* Proceedings of the National Academy of Sciences, 111(37): 13290–13294, 2014. https://doi.org/10.1073/pnas.1403657111

[11] Jean-François Cordeau and Gilbert Laporte. *The Dial-a-Ride Problem: Models and Algorithms.* Annals of Operations Research, 153(1): 29–46, 2007. https://doi.org/10.1007/s10479-007-0170-8